Jordan Strine, Amal Allas A Alqahtani, Ngoc Lan Anh Nguyen

Project 1

CPS 356

10/12/2026

Problem 1: Classic CPU Scheduling Algorithms

Problem 1 pertains to comparing a variety of scheduling algorithms used in operating systems. The four algorithms implemented in this assignment were:

- First-Come, First-Served (FCFS)
- Shortest Job First (SJF)
- Shortest Time-to-Completion First (STCF)
- Round Robin (RR)

Before beginning the implementation, I started to plan out what complex data types may be of use in this case so that I could minimize how much code I was repeating. In helpers.c I created structs for processes to track our relevant information, nodes to house a pointer to a process and allow us to have a linked list to serve as our source of truth for the algorithms to reference, and a queue struct to serve as the storage for our processes once the algorithms had begun to schedule them for execution. I created an algorithm struct as well, as the 4 methods used basically all the same properties. Some helper functions were added as well, and I can think of half a dozen more that would've reduced redundancy among the implementations but the three I made were very helpful. The freeQueue method allowed me to pass in any of the Queues I created and have it handle freeing the memory. The two vital helpers were the two gantt chart functions that handled outputting the gantt charts for all the algorithms as well as the information asked for in the assignment such as: execution order, waiting time per process, turnaround time per process, average waiting time, and average turnaround time. I created two gantt chart functions because two of the algorithms are preemptive (STCF and RR), while the other two are not (FCFS and SJF). This created some complexity as the preemptive methods meant that processes could be placed on the queue multiple times which needed to be handled differently than the nonpreemptive methods where processes ran to completion upon entering the queue.

Each of the algorithms offers different benefits and downsides depending on the aim of the developer. Because they queue processes based on varying factors, a processes turnaround time and waiting time can vary vastly between the different methods. The first algorithm to be implemented was First-Come, First Served as it had

the simplest logic to implement and would serve as the blueprint for more complex logic in the coming methods. The user is prompted to provide N processes, where N is a number of their choosing. Processes are then generated with a pid and arrival time according to what order they were entered into the program. The user inputs a run time for each process to generate our linked list of processes which are then referenced by the algorithms to begin building their respective queues. The FCFS algorithm traverses the list N times and tracks which process has the earliest arrival time, and enters it into the queue if it has not already been queued. An array was used to track what processes had been queued for easy lookup. A helper method for inserting the new nodes into the queue would have probably been a good idea as it repeats across the 4 variations.

Shortest Job First was the second algorithm to be implemented as it built off FCFS with the least amount of change. The handling of the queue and node traversals are functionally the same, however instead of arrival time as the criteria it queues whichever process has the shortest runtime. Since we might be jumping around to processes that haven't technically "arrived" into the scheduler, we also add a check to make sure that the time that has elapsed is greater than the arrival time of a process so we aren't queuing a process prematurely. To do this we use the timeElapsed field of the algorithm struct to serve as a "clock". This implementation was straightforward after FCFS and uses the same gantt chart helper function as FCFS.

Shortest Time to Completion First was the third algorithm and the first of the set of preemptive methods, and introduced some increasingly complex logical handling. Since we're no longer guaranteed to finish a process when it has been queued, we now loop through the list of processes until each of them has been finished. Adding to the logic we introduced in SJF where we checked that a process wasn't accessible before it "arrived", we also add a check to see if a process has any remaining time to be completed. If a process has been completed, it obviously doesn't need to be queued again. The criteria for giving a node precedence to be queued now revolves around the shortest remaining time value among all nodes.
Each iteration of our loop counts as a duration of 1, so each iteration will decrement the remaining time value of the selected node until it hits 0 and finishes, or another node arrives that has a lower remaining time. This is the first of the methods to use the preemptive gantt chart function, and in essence follows the same logic as the other function but formats the output with consideration of a process having multiple entries in the queue. Each entry will also display the duration that the node ran for as duration is tracked when generating the queue.

Round robin was the final algorithm to implement and was extremely intensive with the logic required to get it to function. The chief cause of this extra planning was

that we needed to maintain two separate queues for the logic required. One queue functioned exactly as the STTCF, we would add entries to the queue according to the criteria we implemented, but a second queue was needed to maintain the order of the scheduling. Since round robin gives each process a set amount of time and then moves on to the next, we needed to keep a cycling queue in sync with the job queue. When a job is added to the job queue, the front process of the cycling queue has its remaining time decremented by the time span value. The front then was moved to the tail if it had remaining time and the cycle repeated.

Since I made the process information somewhat dynamic I was able to test a variety of data sets and make sure the use cases were all covered based on what the input specifications are. The biggest challenges were the constant handling of multiple pointers and the logic cycles necessary to maintain usability. Becoming familiar with C as a language would also have sped up my development time as well, as I kept defaulting to OOP paradigms like classes, which structs are definitely not. I completed the entirety of problem 1 on my own.

Problem 2: Multi-Level Feedback Queue (MLFQ)

To Be Done By Ngoc Lan Anh Nguyen

For problem 2, I implement the multi-level feedback queue scheduler using a C program. For the Multi-level feedback queue, there are 5 rules we have to maintain.

First, I assume that the time period is the 50s and the time quantum is 5s. Then, I declared variables that would be used in the program as time management variables (for example: currentTime, done, etc.) and Process with pID, arrivalTime, etc. Then, the program asks users to enter the number of queues and processes. Based on that, there is a loop to assign multiple processes with pID, arrivalTime, and burstTime into an array. Next, I implemented a main loop using the while condition to check if the completed process is equal to the total processes entered at the beginning. If yes, the program finishes and prints out the Gantt chart. If not, based on the priorities, the CPU will be assigned to different processes until every job is done.

Within the loop, we first check if there are any new arrivals processes coming during the execution. If yes, increase its orderInQueue by 1. Then, to prevent starvation, we set a time period of 50s. After 50s, the program moved all jobs back to the highest priority queue.

```
//While loop ends when completes all jobs
while (done < n) {

    //Check for new arrivals
    for ( i = 0; i < n; i++) {
        if (p[i].arrivalTime == currentTime && p[i].isDone == false) {
            p[i].orderInQueue = ++counter;
        }
    }

    //After 50s, boost all processes to highest priority
    if (currentTime>0 && currentTime % BOOST == 0) {
        for ( i = 0; i < n; i++) {
            p[i].priorityQueue = 0;
        }
    }
```

This part is to satisfy rule 1 and 2 of the MLFQ scheduler. The first is used to interrupt the CPU execution when there is a new arrival process. As any new process arriving is assigned as the highest priority queue, the current running process has to release the CPU. The second if condition is used to decide which processes have the highest priority among the processes. It loops through every process and decides either the first arrived processes or the processes with highest priority queues or if there are 2 processes having the same priority queues, it keeps checking on the one with smaller order to use the Round-Robin.

```
//If cpu is used, check if any new arrival processes with higher priority
if ( run != -1) {
    for ( i = 0; i < n; i++) {
        if (i != run && p[i].arrivalTime <= currentTime
            && p[i].isDone == false && p[i].priorityQueue < p[run].priorityQueue) {

            p[run].orderInQueue = ++counter;
            run = -1;
            break;
            }
    }
}

//Situtation when cpu is free, decide which process takes it
if (run == -1) {
    int highestPriority = -1;
    for (i = 0; i < n; i++) {
        if (p[i].arrivalTime <= currentTime && p[i].isDone == false) {
            if (highestPriority == -1 || p[i].priorityQueue < p[highestPriority].priorityQueue
                || ( p[i].priorityQueue == p[highestPriority].priorityQueue
                && p[i].orderInQueue < p[highestPriority].orderInQueue )) {
                highestPriority = i;
                }
        }
    }
    run = highestPriority; //process with highest priority takes over CPU
    used = 0; //reset time quantum
}
```

Otherwise, we also have to maintain the time management.This works to assign the processes using CPU at each time unit. Also, to track for the remaining burst time and the time used of each process. The variables used to check if they reach the time allotment or not. Then, the next condition is used to check which processes are doing their job or not and to track if processes used up all their time allotment. If yes, its priority queue will decrease. If there are two processes coming to the queue at the same time, the orderInQueue is used to keep track of which processes are coming first.

```
//Time management
if (run == -1) {
    timeline[currentTime] = 0;
    qLevelTimeline[currentTime] = -1;
} else {
    timeline[currentTime] = p[run].pID;
    qLevelTimeline[currentTime] = p[run].priorityQueue;
    p[run].remainingTime--;
    used++;
}
currentTime++;

//Check if processes finished
if (run != -1) {
    if (p[run].remainingTime == 0) {
        done++;
        p[run].isDone = true;
        run = -1;
    } else if (used == QUANTUM) { //Check if processes used all time quantum, increase its priority queue
        if (p[run].priorityQueue < q - 1) {
            p[run].priorityQueue++;
        }
        p[run].orderInQueue = ++counter;
        run = -1;
    }
}
```

For the given example for problem 2, here is the Gantt chart following the Multi-level feedback queue. Input of queues is 5, processes is 4.

```
C:\Users\anhng\CLionProjects\Problem2_Draft\cmake-build-debug\
Enter number of queues:5

Enter number of processes:4

Enter pID, arrival time, and burst time:1 0 8

Enter pID, arrival time, and burst time:2 1 4

Enter pID, arrival time, and burst time:3 2 9

Enter pID, arrival time, and burst time:4 3 5

Gantt chart:
| P1 | P2 | P3 | P4 | P1 | P3 |
0    5    9   14   19   22   26
 Q0 | Q0 | Q0 | Q0 | Q1 | Q1 |
Process finished with exit code 0
```

Problem 3: Multiprocessor Scheduling with Work Stealing

Amal Alqahtani

In Problem 3, a work-stealing scheduling simulation will be implemented in multiprocessors through an algorithm known as work stealing. The main objective of the problem lies in learning how multiple CPUs can handle their respective processes and how work can be done among those CPUs if some of them are busy whereas the others are idle. It is worth noting that in multiprocessors, each CPU has a set of processes to run and the amount of work that each one of them gets may not be equal. This can result in one CPU finishing its work while another CPU still has processes waiting. Work stealing helps address this situation by allowing an idle CPU to take a waiting process from another CPU's ready queue.
In this part of the project, the simulation code was programmed with the help of C programming language and run in CLion IDE. The code takes number of CPUs, number of processes, and details of the processes such as process ID, arrival time, and CPU time required as inputs. There is a provision for up to 2 to 8 CPUs and up to 30 processes in the program. Moreover, the program also checks whether the inputs are within the required range and process ID is unique.

Planning and Data Structures

In order to implement the scheduling algorithm, there was the need for storage of data regarding the processes and CPUs. The two structures used in C included Process and CPU.
The Process structure holds some important values like the process ID, arrival time, run time, initially assigned CPU, start time, completion time, and remaining time for each process. All of these values are used in order to calculate when the process can become available, when the process will start execution, and when the process completes execution.

The CPU structure holds some useful values related to the CPUs themselves. There is an array of the ready queue, which is used to keep the information of the processes that have been added to the ready queue but have not started executing yet, a number of processes waiting in the ready queue, and a process ID of the process currently executing on the CPU.

Moreover, I designed two functions that could be described as helper ones. Namely, the enqueue () function and dequeue() function were created. The first one puts a new process at the end of the ready queue, while the second one takes the first waiting process out of it.

Scheduling Algorithm

The scheduling algorithm starts by getting the number of CPUs and information of the processes. Processes are then allocated into the CPUs by implementing a round-robin allocation technique. When there are three CPUs, the first process is allocated to CPU 1, second process to CPU 2, third process to CPU 3, and fourth process to CPU 1. This pattern continues until all processes have been assigned.
When processes have been allocated, the simulation starts by using a clock set to zero. At each time unit, the program checks whether any new processes have arrived. In case where a process has arrived at its arrival time, the process is inserted into the ready queue of the initially assigned CPU.
The FCFS scheduling policy is applied to the ready queue of each CPU. Under this scheduling policy, the first waiting process is executed before those that follow it. When a process starts executing, the process completes execution without getting interrupted.
The simulation works in a loop until all processes terminate. In every loop iteration, the process performs the following tasks:

- Checking if there are any processes that have just arrived
- Scheduling local work on idle CPUs
- Performing work stealing when necessary
- Simulating the execution of one-time unit.

The remaining execution time of each running process decreases by one. When a process's remaining time reaches zero, its completion time is recorded and the CPU becomes available for another process.

Work-Stealing Implementation

The most important feature of this problem is the work-stealing mechanism. After each CPU has been given an opportunity to run a process from its own ready queue, the program checks whether any CPU is still idle.

If an idle CPU has no local waiting processes, the program searches the ready queues of the other CPUs. This involves comparing the number of processes in the respective queues and determining which CPU has the highest number of processes in the ready queue. When there is a waiting process, the idle CPU takes out the process from the queue and executes it.
For instance, when CPU 1 is processing a process but also has a waiting process and CPU 2 has finished all its processes locally, CPU 2 can steal the process from CPU 1. This prevents CPU 2 from remaining idle when work is available elsewhere.

The program also keeps track of successful and unsuccessful stealing attempts. A successful steal occurs when an idle CPU finds and takes a waiting process from another CPU. An unsuccessful attempt occurs when an idle CPU checks for waiting work but none is available. In this implementation, an idle CPU may make another unsuccessful attempt during a later simulated time unit.

Testing and Results

After implementing the scheduling logic, I tested the program in CLion using three CPUs and five processes. The following test case was used:

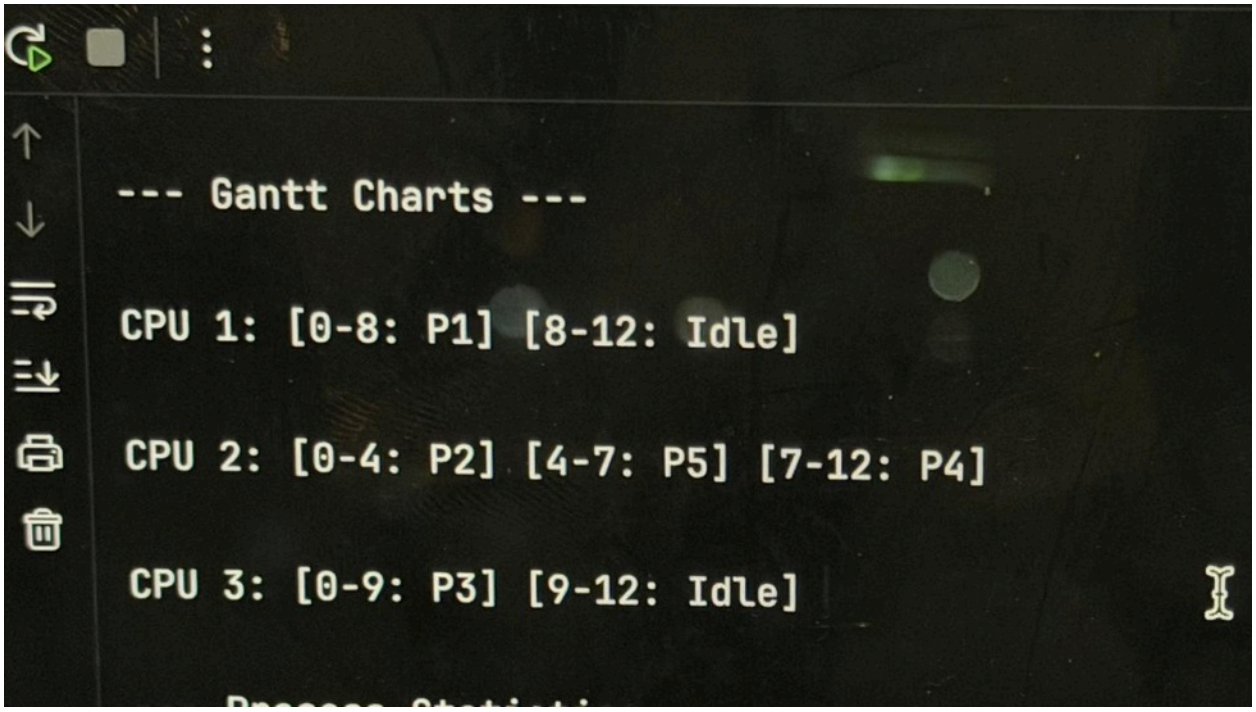
Process	Arrival Time	Run Time
P1	0	8
P2	0	4
P3	0	9
P4	1	5
P5	2	3

P1 and P4 were allocated to CPU 1, P2 and P5 were allocated to CPU 2, and P3 was allocated to CPU 3.

At time 0, processes P1, P2, and P3 started execution on their allocated CPUs.P4 arrived at time 1 and entered CPU 1's waiting queue because P1 was still running. P5 arrived at time 2 and waited in CPU 2's queue while P2 was running.

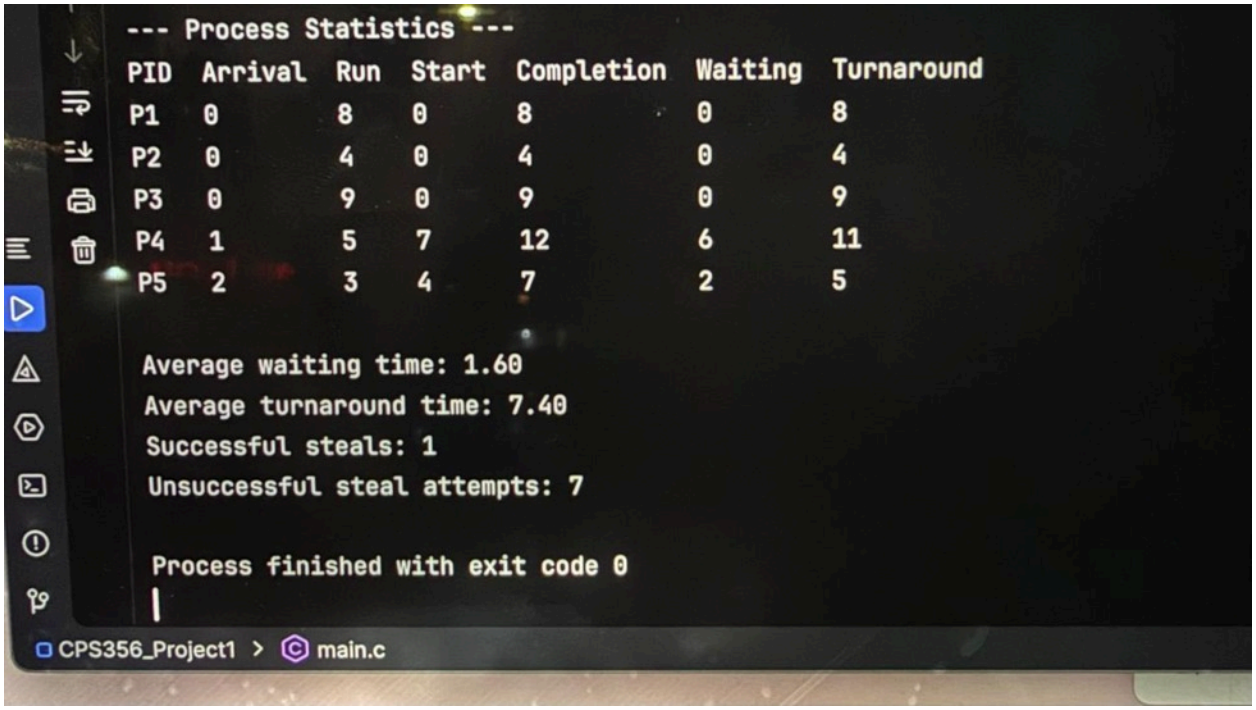
At time 4, P2 completed, allowing CPU 2 to begin executing P5. P5 completed at time 7. At this stage, CPU 2 had exhausted all the processes in its own ready queue, but process P4 remained waiting in the ready queue of CPU 1. CPU 2 succeeded in stealing process P4 from CPU 1 and processed it from time 7 till time 12.

The resulting Gantt chart showed the following execution intervals:



The simulation also calculated the waiting time and turnaround time of each process. Turnaround time was calculated by subtracting the arrival time from the completion time. Waiting time was calculated by subtracting the process's execution time from its turnaround time.

The results for this test case were:



These results demonstrate that the work-stealing mechanism operated as expected for the sample input. In particular, CPU 2 was able to execute P4 while CPU 1 was still completing P1. Under the same initial assignment without stealing, P4 would have waited until time 8 to start on CPU 1. With work stealing, it started at time 7 instead.

Challenges and Improvements

One part of this project was understanding how to manage several simulated CPUs and their ready queues while keeping track of process arrival and execution times. The scheduling logic needed to handle new arrivals, local process selection, work stealing, and completion in the correct order so that the Gantt chart and calculated statistics would be consistent.

I also encountered some development-environment issues while setting up and running the program in CLion, particularly with the CMake configuration on macOS. After resolving the configuration issues, I was able to compile and run the program successfully and verify the sample results.

One limitation of the current implementation is that it is a discrete-time simulation rather than a program that creates actual parallel CPU threads. A virtual clock is used to

simulate the functioning of several CPUs. Non-preemptive scheduling technique is applied here as well; this means that when a process starts executing, it continues until completion.
In the future, this algorithm could be enhanced through testing various process configurations, comparing execution results both with and without work-stealing technique, and experimenting with ways of choosing from which CPU to steal work. These improvements would help evaluate the scheduling approach more thoroughly.

Conclusion and Contribution

For Problem 3, I worked on implementing the multiprocessor scheduling simulation in C, including the process and CPU structures, ready queue operations, scheduling loop, work-stealing mechanism, Gantt chart output, and performance calculations. I tested the implementation in CLion and shared the source code through our GitHub repository workflow.
Overall, this problem helped me gain a better understanding of CPU scheduling, process management, and how work stealing can allow idle CPUs to execute processes waiting in other CPU queues. The sample test confirmed that the implementation could perform a successful work steal and generate the required scheduling statistics..